

Day 45: Display Account Method

Day 45 — Display Account Method

1. Day Number

Day 45

2. Topic Name

Methods

Today you will learn how to add a **method** inside a Python class.

A method is simply a function that belongs to a class.

3. Connection to Yesterday

Yesterday, you created a `BankAccount` class using a constructor:

```
BankAccount
├── account_holder
└── balance
```

Today, you will add behavior to that class.

You will create a method called:

```
display_account()
```

Its job will be to display the account holder name and balance.

Conceptually:

```
Yesterday:

BankAccount
├── account_holder
└── balance

Today:

BankAccount
├── account_holder
├── balance
└── display_account()
```

4. Important Topics

Today's important concepts are:

- method
 - self
 - object method call
 - accessing object attributes inside a method
-

5. Foundational Notes

What is a method?

A **method** is a function written inside a class.

Normal function:

```
def greet():  
    print("Hello")
```

Method inside a class:

```
class Example:  
    def greet(self):  
        print("Hello")
```

The major difference is that a method normally receives:

```
self
```

as its first parameter.

What does self mean?

self refers to the **current object**.

Suppose an object contains:

```
name = "Amit"
```

Inside a method, you can access that object's name using:

```
self.name
```

For your bank account, you will have attributes similar to:

```
self.account_holder  
self.balance
```

So the method can access the information that was stored when the object was created.

Constructor vs Method

A constructor initializes the object:

```
__init__()
```

A normal method performs some action:

```
display_account()
```

Think of it like this:

```
Constructor
  ↓
creates/stores object data

Method
  ↓
uses that object data
```

For example:

```
Create account
  ↓
BankAccount object
  ↓
name = Ravi
balance = 5000
  ↓
display_account()
  ↓
prints account details
```

How do we call a method?

Suppose we have an object:

```
student = Student(...)
```

A method can be called using:

```
student.show()
```

General pattern:

```
object.method()
```

For today's task, the structure will look conceptually like:

```
bank_account_object.display_account()
```

Notice that you do **not** manually pass `self`.

Python handles that automatically.

6. Easy Example

Here is a simpler example using a Product class:

```
class Product:
    def __init__(self, name):
        self.name = name

    def display(self):
        print(self.name)
```

Create an object:

```
product1 = Product("Laptop")
```

Call the method:

```
product1.display()
```

Output:

```
Laptop
```

What happened?

When:

```
product1.display()
```

is called, Python understands that:

```
self
```

refers to:

```
product1
```

Therefore:

```
self.name
```

means:

```
product1's name
```

7. Problem Statement

Create a BankAccount class.

The class should already have:

- account holder name
- balance

stored using its constructor.

Now add a method named:

```
display_account()
```

The method should print:

- account holder name
- balance

Then create one `BankAccount` object and call the method.

8. Concepts Used

You will use:

Class

Used to define the structure of a bank account.

```
BankAccount
```

Constructor

Used to initialize account information.

```
__init__()
```

Attributes

Used to store data belonging to an object.

Conceptually:

```
account_holder  
balance
```

Method

Used to perform an action using object data.

```
display_account()
```

self

Used inside the method to access the current object's attributes.

Object

A real instance of the `BankAccount` class.

Method call

Used to run the method:

```
object.display_account()
```

9. Thought Process

Before writing code, think through the problem.

Step 1: What object are we representing?

A bank account.

Therefore, we need:

```
BankAccount class
```

Step 2: What information does the account contain?

Two pieces of data:

```
account holder name  
balance
```

These should already be stored as object attributes.

Step 3: What new behavior do we need?

We want the object to display its own information.

Therefore, create:

```
display_account()
```

Step 4: How can the method access the object's information?

Using:

```
self
```

Conceptually:

```
self.account_holder  
self.balance
```

Step 5: How do we run the method?

Create an object and call:

```
object.display_account()
```

Overall flow:

```

Define BankAccount
    ↓
Create constructor
    ↓
Store account holder
    ↓
Store balance
    ↓
Create display_account method
    ↓
Access attributes using self
    ↓
Create BankAccount object
    ↓
Call display_account()
    ↓
Print account details

```

10. Pseudocode

```

START

Create BankAccount class

    Create constructor with account holder and balance

        Store account holder in object
        Store balance in object

    Create display_account method

        Print account holder
        Print balance

Create one BankAccount object

Call display_account method using the object

END

```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Keep both the **data** and the **behavior related to that data** inside the BankAccount class.

Conceptually:

```

BankAccount
|
├── Data

```



This is one of the main ideas behind Object-Oriented Programming.

Instead of writing something separate like:

```
external function → bank account data
```

you let the object manage its own behavior:

```
BankAccount object
    ↓
display_account()
```

12. Easy Edge Cases

Edge Case: Balance is 0

Example:

```
Account Holder: Ravi
Balance: 0
```

A balance of 0 is valid.

Do not treat:

```
0
```

as an error for today's exercise.

Your method should still display the account normally.

13. Expected Input

For this exercise, you do not need `input()` unless you want extra practice.

You can create the object using values directly.

Conceptually:

```
Account holder: Ravi
Balance: 5000
```

The object would contain:

```
BankAccount Object
|
```



```
|— account_holder → "Ravi"  
|— balance → 5000
```

14. Expected Output

For an account with:

```
Account holder = Ravi  
Balance = 5000
```

the output could look like:

```
Account Holder: Ravi  
Balance: 5000
```

For the edge case:

```
Account holder = Ravi  
Balance = 0
```

expected output:

```
Account Holder: Ravi  
Balance: 0
```

The exact formatting can vary slightly as long as both values are displayed correctly.

15. Hint Only

Inside your existing BankAccount class, add another function below `__init__()`.

Remember that the method should receive:

```
self
```

Think about which two attributes you created yesterday and access them through:

```
self.<attribute>
```

Finally, after creating your object, remember the general method-call pattern:

```
object_name.method_name()
```

Do not create another separate function outside the class for displaying the account details. The goal today is to practice making the behavior part of the BankAccount object itself.